⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Status: Documented — reflexive. Clause: 7 — Software Risk Management · **Register:** Document Register

> This is also the one document in the set that most needs its framing read first: **this project carries no patient risk.** What follows applies Clause 7's discipline to a different, real harm this project *can* cause — teaching a reader something about IEC 62304 that is wrong.

What the standard requires (Class C — every live sub-clause of Clause 7)

`7.1.5` and `7.3.2` were deleted by Amendment 1 and are excluded below — see the site's own Safety Class Applicability section.

REF	TITLE	APPLIES AT	OUTPUT
7.1.1	Identify software items that could contribute to a hazardous situation	**B, C**	—
7.1.2	Identify potential causes of contribution to a hazardous situation	**B, C**	—
7.1.3	Evaluate published SOUP anomaly lists	**B, C**	—

REF	TITLE	APPLIES AT	OUTPUT
7.1.4	Document potential causes	**B, C**	Potential causes documented in the risk management file
7.2.1	Define risk control measures	**B, C**	Documented risk control measures for each case where a software item could contribute to a hazardous situation
7.2.2	Risk control measures implemented in software	**B, C**	Risk control measures in the requirements, with a safety class assigned to each implementing item
7.3.1	Verify risk control measures	**B, C**	Documented verification, and a review of whether the measure could itself cause a new hazardous situation
7.3.3	Document traceability	**B, C**	Traceability from hazardous situation → software item → software cause → risk control measure → verification
7.4.1	Analyse changes with respect to safety	**A, B, C**	Analysis of changes, including SOUP, for additional causes and risk controls needed — **the only Clause 7 requirement reaching Class A**
7.4.2	Analyse impact of changes on existing risk control measures	**B, C**	Analysis of whether changes could interfere with existing risk control measures
7.4.3	Perform risk management activities based on analyses	**B, C**	—

Reframing the hazard

A conventional risk management file identifies hazards to a *patient*. This project has none. What it does have: a reader — plausibly a real software engineer, quality engineer, or regulatory affairs professional, per this course's own stated audience — who may carry what they learn here into a real medical device project. **The**

hazardous situation this file tracks is: a reader forms an incorrect belief about what IEC 62304 requires, and acts on it in a real regulatory context. That is not a hypothetical for this project — it has already happened once, which is what makes this document able to cite real evidence rather than invented examples.

7.1.1–7.1.2 — Identified software items and causes (a real case)

The defect, as this project's own design notes and README both record: the original data model held one hand-maintained list of safety classes per *clause*. Three errors were found when it was checked line by line against the standard's normative text:

1. **Clause 7 (Software Risk Management)** was recorded as Class B and C only. Filtering the Learn page to Class A hid the entire clause — implying Class A software needs no risk management. **This is backwards**: §7.4.1 applies to every class, and the classification itself is an *output* of risk analysis, not something decided before one.
2. **Clause 5.3 (Architecture)** was recorded as reaching Class A when it has no Class A requirement at all.
3. **Clause 5.4 (Detailed Design)** was recorded as Class C only, when §5.4.1 (subdividing into units) actually reaches Class B.

The software item that could contribute to the hazardous situation: the single clause-level `classes` list in `phases.json`, and the Learn page's filter logic that rendered directly from it.

The potential cause: a hand-maintained summary has nothing to check itself against — a single list can be wrong with no internal signal that it is, which is exactly what let three independent errors persist simultaneously.

7.1.3 — SOUP anomalies

The project's SOUP is Playwright and axe-core, both test-only (see 04 — Architecture). A defect in either could not itself mislead a visitor — neither ships to the browser — but it could produce a **false negative** in the test suite (a real defect

that the tool fails to catch) or a **false positive** (a fabricated failure that wastes investigation time). Both tools are pinned to a specific version (`tests/requirements.txt` ; the `AXE_CDN` URL) specifically so a version change is a deliberate, reviewable decision rather than something that happens silently on the next run.

7.2.1–7.2.2, 7.3.1 — Risk control measures, defined, implemented, verified

Two controls were put in place, not one — deliberately, since a single control reintroduces the single-point-of-failure that caused the original defect:

1. **A cross-file invariant**, implemented in `learn.js` 's `mergeApplicability()` : the clause-level `classes` in `phases.json` must equal the union of that clause's sub-clause classes in `applicability.json` . If they disagree, the page **refuses to render** and names the offending clause — see 04 §5.3.5 for why this counts as the architectural segregation Class C's 5.3.5 asks for. A loud failure was chosen deliberately over a page that quietly teaches something wrong.
2. **A regression test.** The `applicability` group in `tests/test_site.py` puts the original, incorrect Clause 7 value back and asserts the site rejects it — so the specific defect that happened once cannot silently return.

Could the control itself create a new hazardous situation, as 7.3.1 requires asking? Yes, one was considered: an overly strict invariant could make the site refuse to render on a *correct* but differently-shaped update to the data, denying a reader access to real content over a false alarm. The mitigation is that the invariant checks a specific, well-defined mathematical relationship (set union), not an arbitrary shape rule, and is itself covered by tests asserting it accepts valid data.

7.3.3 — Traceability

HAZARDOUS SITUATION	SOFTWARE ITEM	CAUSE	RISK CONTROL	VERIFICATION
Reader concludes Class A software needs no risk management	`phases.json` clause-level `classes`; Learn page filter	Single hand-maintained list, no internal cross-check	Cross-file invariant in `mergeApplicability()`; regression test	`applicability` test group
Reader concludes Clause 5.3 has a Class A requirement	Same	Same	Same	`applicability` test group ("Clause 5.3 has no Class A requirement at all")
Reader concludes Clause 5.4 is Class C only	Same	Same	Same	`applicability` test group (5.4.1 spot check)

7.4.1–7.4.3 — Ongoing change analysis (the requirement that reaches Class A)

Every subsequent change to `applicability.json` is required to pass the same invariant and the same regression test before it can be accepted — so 7.4.1's "analyse changes with respect to safety" is enforced mechanically for this specific file, not left to a reviewer's memory. This is also why §7.4.1 is singled out in 01 — General Requirements: it is the one Clause 7 requirement that would still matter even under the literal (out-of-scope) classification this project would actually receive.

Escalations from the problem log

7.3.3's traceability chain ends at "verification" for a hazard already known about. What closes the loop for a hazard found *later*, by a test that starts failing, is 13 — Problem Resolution's anomaly log — but only if a defect found there is actually

connected back to this file, rather than living out its whole life as a line in a CSV nobody with risk-management responsibility ever reads.

So every check in `tests/test_site.py` that verifies something in this file's hazard model — the safety-class mapping, the deliverables data derived from it, and the cross-file invariant and regression test that are this file's own risk controls (§7.2.1–7.3.1 above) — is tagged `risk=True`. Any anomaly raised by one of those checks is escalated here automatically, by id, for as long as it stays open:

No risk-of-harm anomalies currently open — see `tests/anomaly_log.csv` for the full history.

This section is **generated**, by `escalate_risk_anomalies()` in `tests/test_site.py`, every time the suite runs — the table above (or the "none open" line) is rewritten in full each time, the same way the anomaly log itself is. Do not hand-edit between the markers; a manual change there is overwritten on the next run. Which checks carry the `risk=True` tag is the thing to edit instead — see the `Results.group()` / `Results.check()` docstrings in `tests/test_site.py` for the exact criterion, and the site's own Anomaly log section for how the tagging and escalation mechanics work together.

Gaps

- This risk management file covers **one** category of hazard (incorrect regulatory content) in depth, because it is the one this project actually has real evidence for. A conventional risk management file would need a broader hazard-identification pass across the whole product; that has not been performed here, and this document should not be read as claiming it has.
- Residual risk is not formally signed off by anyone independent of the person who implemented the controls — for a real Class C project this would be a critical finding.